

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 5 — Skills

Skill e Claude Code

M5-D03 Tre garanzie

Demo del docente in Claude Code: tre garanzie, quattro coppie, un controllo

Una skill è una cartella con uno `SKILL.md`; il contorno lo promette Claude Code. Le frasi di «Extend Claude with skills» e per ognuna una **coppia** di esecuzioni diverse in una cosa sola:

che cosa dice Anthropic	la coppia, e che cosa guardare
«The command output replaces the placeholder, so Claude receives actual data»	`codice`: `cat codice.txt` nel corpo, con il `!` e senza. Il dato arriva **senza strumenti**?
«… when a command's permission check returns anything other than allow, Claude Code aborts the invocation»	`comando`: `bash conta.sh` con il `!`, con `allowed-tools` e senza. Gira, o **si ferma**?
«The grant clears when you send your next message.»	`permesso`: `bash conta.sh` nel corpo, con il campo e senza; poi un messaggio. **Negato**?
«The subagent doesn't see your conversation history»	`parola`: una parola in codice; `/prova-parola`, senza `context: fork` e con. **Torna**?

`!`: un comando in una riga; l'output prende il suo posto. `allowed-tools`: strumenti senza chiedere, per un turno. `context: fork`: gira in un subagent. Undici chiamate `claude -p` in `dontAsk`, che nega ciò che chiederebbe un permesso. Uno script legge gli stream e decide

```
# nello spazio di lavoro di M5-D03, da tools/prepara_esercizio.sh
bash in_classe.sh
```

Indovinate, a voce: la skill con `context: fork` vedrà la parola in codice detta prima nella conversazione?

Nella registrazione del corso: **no**. Con `context: fork` la risposta è «NON LA VEDO», senza è «ciaspola-2078b9», la parola: «confermata 3 su 3»

M5-D03 Tre garanzie

Demo del docente in Claude Code: tre garanzie, quattro coppie, un controllo

la coppia codice: la riga che cambia, con e senza

```
Il codice di controllo è: !`cat codice.txt`
Il codice di controllo è: `cat codice.txt`
```

il controllo, a capo per la slide: «[…]» segna un taglio

```
[…] ! porta il dato prima che Claude legga (codice, 1): confermata
  (con ! il codice nella risposta, nessuno strumento; senza !:
  Bash (cat codice.txt), il codice c'è)
Un comando non consentito ferma l'invocazione (comando, 1):
  confermata (con allowed-tools la sentinella nella risposta,
  nessuno strumento; senza, fermata con zero turni)
[…]
! porta il dato prima che Claude legga: confermata 3 su 3
Un comando non consentito ferma l'invocazione: confermata 3 su 3
[…]
```

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- Senza il campo, nello stream: «Shell command permission check failed for pattern …». Il rimedio: «pre-approve it with allowed-tools». La sentinella la stampa `conta.sh`, e nel suo testo non c'è. `cat` passa comunque: è di **sola lettura**; `bash conta.sh` no

M5-D03 Tre garanzie

Demo del docente in Claude Code: tre garanzie, quattro coppie, un controllo

il controllo, a capo per la slide: «[…]» segna un taglio

```
[…] permesso, con, il messaggio dopo, 1: risposta «Questa volta
  non sono riuscito a eseguire `bash conta.sh`. […] bash conta.sh:
  negato; […]
parola, inline, 1: risposta «ciaspola-2078b9»; […]
parola, fork, 1: risposta «NON LA VEDO.»; […]
[…]
allowed-tools vale per un turno (permesso, 1): confermata (con il
  campo eseguito; al messaggio dopo negato; senza il campo negato)
Con context: fork il subagent non vede la conversazione (parola,
  1): confermata (senza context: fork la parola torna; con
  context: fork no, e lo stream mostra il subagent)
[…]
allowed-tools vale per un turno: confermata 3 su 3
Con context: fork il subagent non vede la conversazione:
  confermata 3 su 3 […]
```

Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- Il permesso decade «even though the skill content stays in context»
- Il nome inganna: «Despite the name, a skill with context: fork doesn't run in a fork of the current conversation». È un subagent; con `-p`, Claude Code ne aspetta il risultato

Skill e subagent, nei due sensi

La skill come compito di un subagent, o un subagent con le skill già caricate

«Extend Claude Code»: «They can combine. A subagent can preload specific skills (skills: field). A skill can run in isolated context using context: fork.»

	system prompt	il compito	carica anche
Skill con `context: fork`	dal tipo di agente	il contenuto di `SKILL.md`	`CLAUDE.md`, secondo il contesto iniziale dell'agente
Subagent con il campo `skills`	il corpo in Markdown del subagent	il messaggio di delega di Claude	le skill precaricate e `CLAUDE.md`, secondo il contesto iniziale del subagent

La tabella della pagina delle skill di Claude Code, in italiano

- Con il campo `skills`: «The full skill content is injected, not only the description.» Non per le skill con `disable-model-invocation: true`: il precaricamento attinge a quelle che Claude può invocare
- Un subagent normale: «It doesn't see your conversation history, the skills you've already invoked, or the files Claude has already read.»
- Una skill con `context: fork` e `agent: Explore`: il suo contenuto, il system prompt dell'agente, «and none of your CLAUDE.md files»
- «context: fork only makes sense for skills with explicit instructions»: con sole linee guida il subagent non riceve un compito, e torna «without meaningful output»
- Gira in **background**, e si continua a lavorare; `background: false` per aspettare il risultato; con `-p` o con l'Agent SDK, Claude Code aspetta comunque

Skill e hook, e il resto

Una regola che deve valere sempre, e i campi che danno e tolgono

- «Claude skipped a rule that must hold every time: move the rule into a hook.» Il campo `hooks` del frontmatter lo tiene con la skill: si registra quando la skill viene invocata, e resta attivo **per il resto della sessione**
- `disallowed-tools` **toglie**: «Tools removed from Claude's available pool while this skill is active. … The restriction clears when you send your next message.» `allowed-tools` **non restringe**: «It does not restrict which tools are available: every tool remains callable, and your permission settings still govern tools that are not listed.»
- La guida d'esame di Claude Certified Architect scrive «to restrict tool access during skill execution»: vale la documentazione di Claude Code. Per lo standard Agent Skills il campo è sperimentale: «(Experimental)»
- Un dato di una prova, non una regola della documentazione (3 novembre 2026, Claude Code 2.1.317): in `claude -p`, in dontAsk, una skill con `allowed-tools` o `disallowed-tools` che Claude invoca da sé è negata, finché le regole non nominano `Skill(nome)`. Il sintomo: la chiamata in `permission_denials`
- `paths`: Claude carica la skill da sé solo lavorando su file che corrispondono ai pattern. `model` ed `effort`: per il resto del turno corrente

Una skill è istruzioni; Claude Code ne garantisce il contorno: i dati, i permessi, l'isolamento.